

IOT ASSIGNMENT

Appendix - I

The following criteria will be used for final evaluation by the Techible Academic Committee:

Evaluation Area	Weight	Definition of Excellence
Code Integrity	30%	Code is modular, utilizes object-oriented principles, and includes thorough documentation with comments.
Git Proficiency	20%	Features a consistent commit history (minimum 5 commits per module).
System Architecture	30%	Demonstrates accurate schematics and robust edge-case handling.
Presentation	20%	READMEs are high-quality, include clear setup instructions, and contain necessary media.

Q35. REST API – Weather Data Fetcher

Program ESP32 to fetch real weather data from OpenWeatherMap API (free tier). Display on Serial Monitor: City Name, Temperature (API value vs DHT11 local value), Humidity, Weather Description. Compare API data with your local DHT11 reading and print the difference. Store your API key in a separate config.h file (do NOT commit this file – add to .gitignore).

Section C: Protocol Theory (2 marks each)

Q36. MQTT vs HTTP Comparison

Create a table comparing MQTT and HTTP across: Architecture pattern, Data transfer model, Power consumption, Latency, Use case suitability for IoT, and Security considerations. Based on your analysis, which would you choose for a 1000-node smart agriculture deployment and why?

Q37. Wi-Fi Security for IoT

Research and write 250 words on: Common Wi-Fi attack vectors on IoT devices (WPA2 vulnerabilities, evil twin attacks), Best practices for securing ESP32 Wi-Fi connections (certificate pinning, WPA3), and Why hardcoding passwords in IoT source code is dangerous. Reference at least 2 sources in your answer.

Q38. LoRa & LPWAN – Long Range IoT

LoRa (Long Range) is widely used for IoT in agriculture and smart cities. Write 200 words explaining: What LPWAN means, How LoRa achieves long range at low power (spreading factor concept), LoRa vs Zigbee vs Z-Wave comparison, and One real-world deployment example in India.

Q39. (Theory) What is an IoT gateway? Draw a simple diagram showing the relationship between: IoT Sensor Nodes → Gateway → Cloud → User Application. What protocols are typically used at each layer? (2 marks)

Q40. (Theory) Explain QoS (Quality of Service) levels in MQTT: QoS 0 (At most once), QoS 1 (At least once), QoS 2 (Exactly once). In an IoT water level monitoring system, which QoS level would you choose and why? (2 marks)

Module 5: Advanced IoT Projects & Problem Statements

Marks: 20 | Questions: Q41–Q50 | Type: Open-Ended Design Challenges



Problem Statement Guidelines

Each problem statement requires:

1. GitHub repository with organized, commented code
2. README with problem statement, solution approach, hardware list, and circuit diagram
3. At least 5 meaningful Git commits tracking your development
4. Short demo video (60–90 seconds) uploaded to GitHub or linked in README

Problem Statement Type A: Smart Home & Environment

Q41. Smart Home Air Quality Monitor

Problem Statement

A family in a Milan(Europe) home is concerned about indoor air quality during winter when they burn wood for heating. Design an IoT-based Air Quality Monitoring System that continuously measures CO/smoke levels (MQ-2 sensor), temperature and humidity (DHT11), and alerts residents when air quality drops below safe thresholds. The system must work without the internet when the router is down.

Requirements:

- MQ-2 gas sensor integration with analog threshold comparison
- DHT11 for temperature and humidity reading
- RGB LED: Green (safe), Yellow (moderate), Red (danger)
- Buzzer alarm with 3 different tones based on severity level
- Local data logging via Serial Monitor in CSV format
- Send alert over Bluetooth to phone (+2 marks)

Evaluation Focus: Sensor calibration logic, threshold design, alarm escalation.

Wokwi Calibration and schematic to attached and random alert as illustration to sent to the phone (ScreenShot).

Serial Monitor Example

Timestamp, MQ-2 Value, DHT11 Temp, DHT11 Humidity, Severity Level

00:01, 220 ppm, 23.4°C, 49%, *SAFE*

00:02, 280 ppm, 23.5°C, 50%, *SAFE*

00:03, 420 ppm, 23.6°C, 49%, *MODERATE*

00:04, 710 ppm, 23.8°C, 48%, *DANGER*

Mobile View

SAFE

Gas:210

Temp:22.8C

Humidity:51%

WARNING

Moderate Smoke Detected

Gas=430

Check Fireplace

DANGER!!

Smoke Level Critical

Gas=765

Open Windows

Leave Room if Necessary

Q42. Automated Plant Watering System

Problem Statement

A J&K horticulture farmer wants to automate irrigation for a small saffron farm plot. The system should read soil moisture every 30 minutes and activate a water pump only when the soil is dry. Saffron is sensitive to overwatering — the system must also prevent watering if it rained recently (use LDR to detect low light/overcast sky as a rain proxy and also leverage the use of api also). The farmer must be able to manually override via a button.

Requirements:

- Soil moisture sensor reading with dry/wet/optimal zones
- Relay module controlling water pump (simulate with LED)
- LDR-based rain proxy logic
- Manual override button with 5-minute lockout after manual activation
- Display system status on 16x2 LCD: moisture%, mode, last watered time (millis-based)
- ESP8266 Wi-Fi logging to Google Sheets via IFTTT/ Google Scripts similar

Problem Statement Type B: Security & Access Control

Q43. Smart Door Lock with OTP

Problem Statement

A university hostel wants to replace physical keys with a PIN-based entry system that also generates a One-Time Password. Design a system where a student enters their 4-digit ID on a keypad, the system generates a 6-digit OTP, sends it over Bluetooth to their phone, and only grants access if they enter the OTP within 30 seconds.

Requirements:

- 4x4 keypad for ID and OTP entry
- 16x2 LCD or 0.96 OLED for prompts and masked input (show for digits)
- HC-05 Bluetooth: Send OTP string to connected phone / Alternate use ESP32
- Servo motor: rotate 90° to simulate door unlock
- 30-second countdown timer using millis() (non-blocking) // Don't use delay
- Log all access attempts (success/failure) to Serial Monitor

Q44. PIR-Based Security Camera Trigger

Problem Statement

A small shop in Haryana, Rohtak wants a low-cost security alert system. When motion is detected after shop hours (manually configured time window), the system should sound an alarm, flash lights, and optionally send a notification. Build the motion-detection and alerting module.

Requirements:

- PIR sensor with adjustable sensitivity (via potentiometer mapped to threshold)
- Active buzzer alarm with 3-stage escalation (warning → alarm → urgent)
- LED strip simulation (3 LEDs flashing in sequence)
- Time-window configuration via Serial command (e.g., SET_HOURS 22 06)
- Log motion events with timestamps to Serial Monitor
- ESP32 sends Telegram bot message on motion detection

Problem Statement Type C: Agriculture & Environment

Q45. Smart Greenhouse Controller

Problem Statement

Floriculture farmers in Kashmir growing tulips and roses need automated greenhouse climate control. Design a multi-parameter monitoring and control system that maintains optimal growing conditions: Temperature 18–22°C, Humidity 60–70%, and sufficient light for 12 hours per day.

Requirements:

- DHT11: Monitor temperature and humidity
- LDR: Monitor light levels with running average over 10 samples
- 3 relay outputs: Heater, Ventilation Fan, Grow Light (simulate with LEDs)
- PID-like control logic (simple bang-bang with hysteresis)
- 16x2 LCD: Cycle through all readings every 3 seconds
- BONUS: MQTT publish of all parameters to cloud dashboard

Q46. River Water Level Alert (Flood Detection)

Problem Statement

Given the frequent flooding near the Tawi and Chenab rivers, design a low-cost IoT flood-early-warning sensor node using an HC-SR04 ultrasonic sensor to monitor water level in a model river (a glass tank). Trigger alerts at predefined levels and transmit data wirelessly.

Requirements:

- HC-SR04 mounted above water surface; distance decreasing = water rising
- 3 alert levels: NORMAL (>30cm clearance), WARNING (15–30cm), CRITICAL (<15cm)
- LED + buzzer alerts with different patterns per level
- Rolling 10-sample average to filter ultrasonic noise
- Wi-Fi/BT broadcast of alert level as a simple string message
- Web dashboard showing live level graph with Chart.js

Problem Statement Type D: Healthcare & Wearables

Q47. Heart Rate & SpO2 Monitor (Simulation)

Problem Statement

Using a MAX30102 pulse oximeter module (or simulate with an LDR as a proxy sensor), build a health monitoring prototype that measures pulse rate and blood oxygen saturation, displays values, and alerts on abnormal readings. This is a simulation for learning sensor interfacing, not a medical device.

Requirements:

- MAX30102 via I2C (or LDR analog simulation with smoothed readings)
- Display: LCD or Serial Monitor with readings updated every 2 seconds
- Alert: Red LED if HR < 50 or > 120 bpm, or SpO2 < 94%
- Implement a 5-reading rolling average to smooth noisy data
- Log readings to a /data/health_log.csv file reference in GitHub README

Open-Ended Design Questions (Theory + Design)

Q48. Design Challenge: IoT for Jammu Smart City

J&K Government is planning a Smart City project for Jammu. As an IoT engineer, propose a system to monitor and manage 3 of the following (choose any 3): Street lighting, Traffic flow, Parking availability, Air quality, Water supply pressure, Waste bin fill levels. For each:

- Identify the sensor(s) needed
- Identify the microcontroller / connectivity module
- Describe the data flow: Sensor → Protocol → Cloud → Dashboard
- Estimate approximate cost per node (in INR)

Present your answer as a structured proposal (500–700 words) uploaded as /final_project/smart_city_proposal.md in GitHub.

1. ***Write the code for each problem separately.***

Q49. Security Vulnerability Analysis

You are given a poorly written IoT code snippet that connects an ESP8266 to a public Wi-Fi network. The code hardcodes the SSID and password, uses unencrypted HTTP (not HTTPS), publishes sensor data to a public MQTT broker without authentication, and never validates incoming commands. Identify at least 5 security vulnerabilities, explain the risk of each, and provide corrected code for at least 2 vulnerabilities.

Q50. Reflection & Learning Portfolio

Describe what you built and what you learned. How will you apply IoT skills in your field of study or future career? What would you build next — describe your own IoT project idea with sensors, connectivity, and application. (Minimum 400 words, personal and reflective)

Submission Guidelines

IoT Course – Assignment Submission Guidelines

Assignment Requirements

All students are required to complete and submit their IoT assignment by following the instructions below.

Step 1: Prepare the Assignment Document

- Complete all the assignment questions.
- Prepare your answers in a document format (Microsoft Word or PDF).
- Ensure your document is well-formatted, neatly presented, and free from plagiarism.
- Include the following details on the first page:
 - Student Name
 - Enrollment/Roll Number
 - Course Name: Internet of Things (IoT)
 - Assignment Title
 - Submission Date

Step 2: Upload the Assignment to GitHub

After completing the assignment, upload the following files to your GitHub repository:

1. **The completed assignment document** (PDF or Word document containing your answers).
2. **A separate file containing the assignment questions** (Each question separately, Example : Question46_Water_Level_Alert.ino).

Step 3: Repository Organization

Your GitHub repository must follow the structure below:

GitHub Profile

```
└─ Summer School
    └─ Assignment
        ├── IoT_Assignment.pdf (or .docx)
        └── IoT_Assignment_Questions.pdf
```

Step 4: Verify Your Submission

Before submitting, ensure that:

- The assignment document opens correctly.
- The assignment questions file has also been uploaded.
- Both files are present in the **Summer School** → **Assignment** folder.
- The repository is properly organized and accessible.
- The file names are clear and meaningful.

Submission Checklist

-  Completed assignment document (PDF or Word).
-  Separate file containing the assignment questions.
-  Both files uploaded to GitHub.
-  Repository structure:
 - **GitHub** → **Summer School** → **Assignment**
-  Files are correctly named and accessible.

Note: Submissions that do not include both the completed assignment document and the separate assignment questions file, or do not follow the required GitHub folder structure, may be considered incomplete.